```
import torch
from transformers import AutoModel, AutoTokenizer, T5ForConditionalGeneration, T5Tokenizer
```

Q1. Load a Transformer Model and Tokenizer

```
# Choosing 'bert-base-uncased' and loading it with its tokenizer.
model_name_q1 = "bert-base-uncased"
print(f"\nLoading model and tokenizer for: '{model_name_q1}'...")
```

```
Loading model and tokenizer for: 'bert-base-uncased'...
```

```
# Loading tokenizer
tokenizer_q1 = AutoTokenizer.from_pretrained(model_name_q1)
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 1.16kB/s]
config.json: 100% 570/570 [00:00<00:00, 12.1kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 5.73MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 2.59MB/s]
```

```
# Loading model
model_q1 = AutoModel.from_pretrained(model_name_q1)
print(f"Model and tokenizer for '{model_name_q1}' loaded successfully.")
```

```
ge is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, ins
uggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP
ensors: 100% 440M/440M [00:04<00:00, 32.8MB/s]
 tokenizer for 'bert-base-uncased' loaded successfully.
```

```
# Printing the model name and total number of parameters.
print(f"\n Model Details :-")
print(f"Model Name: {model_name_q1}")
```

```
 Model Details :-
Model Name: bert-base-uncased
```

```
total_params_q1 = sum(p.numel() for p in model_q1.parameters())
trainable_params_q1 = sum(p.numel() for p in model_q1.parameters() if p.requires_grad)
```

```
print(f"Total parameters: {total_params_q1:,}")
```

```
Total parameters: 109,482,240
```

Q2. Simple Summarization using T5

```
# Using T5ForConditionalGeneration and T5Tokenizer
model_name_q2 = "t5-small"
text_to_summarize_q2 = (
    "Natural Language Processing is a part of Artificial Intelligence that focuses on "
    "interaction between computers and human language. It enables applications like "
    "chatbots, translation, and summarization."
)
```

```
print(f"\nLoading model and tokenizer for: '{model_name_q2}'...")
tokenizer_q2 = T5Tokenizer.from_pretrained(model_name_q2)
model_q2 = T5ForConditionalGeneration.from_pretrained(model_name_q2)
print(f"Model and tokenizer for '{model_name_q2}' loaded.")
```

```
Loading model and tokenizer for: 't5-small'...
tokenizer_config.json: 100% 2.32k/2.32k [00:00<00:00, 41.8kB/s]
spiece.model: 100% 792k/792k [00:00<00:00, 8.27MB/s]
tokenizer.json: 100% 1.39M/1.39M [00:00<00:00, 4.15MB/s]
You are using the default legacy behaviour of the <class 'transformers.models.t5.tokenization_t5.T5Tokenizer'>. This is expected, and simply means
config.json: 100% 1.21k/1.21k [00:00<00:00, 72.4kB/s]
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, in
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTT
model.safetensors: 100% 242M/242M [00:01<00:00, 229MB/s]
generation_config.json: 100% 147/147 [00:00<00:00, 6.05kB/s]
Model and tokenizer for 't5-small' loaded.
```

```
# Prefixing the input with "summarize: "
input_text_with_prefix_q2 = "summarize: " + text_to_summarize_q2
print(f"\nInput text for summarization (with prefix):\n'{input_text_with_prefix_q2}'")
```

```
Input text for summarization (with prefix):
'summarize: Natural Language Processing is a part of Artificial Intelligence that focuses on interaction between computers and human language. It
```

```
# Tokenizing
inputs_q2 = tokenizer_q2(input_text_with_prefix_q2, return_tensors="pt", max_length=512, truncation=True)
print("\nText tokenized.")
```

```
Text tokenized.
```

```
# Generating summary
print("Generating summary...")
summary_ids_q2 = model_q2.generate(
    inputs_q2.input_ids,
    num_beams=4,
    max_length=60,
    min_length=10,
    length_penalty=2.0,
    early_stopping=True
)
print("Summary generated.")
```

```
Generating summary...
Summary generated.
```

```
# Decode
summarized_text_q2 = tokenizer_q2.decode(summary_ids_q2[0], skip_special_tokens=True)
```

```
# Printing the final summarized output
formatted_summary_output = summarized_text_q2.replace(". ", ".\n") # I have used this because one single sentence was looking too long
print("\n Summarized Output :- ")
print(formatted_summary_output)
```

```
 Summarized Output :-
natural language processing focuses on interaction between computers and human language.
it enables applications like chatbots, translation, and summarization.
```

Q3 : Print Number of Layers and Attention Heads in BERT

```
model_name_q3 = "bert-base-uncased"
```

```
# Loading the bert-base-uncased model
print(f"Loading model: '{model_name_q3}' ...")
model_bert_q3 = AutoModel.from_pretrained(model_name_q3)
print(f"Model '{model_name_q3}' loaded successfully.")
```

```
Loading model: 'bert-base-uncased' ...
Model 'bert-base-uncased' loaded successfully.
```

```
# Accessing the model configuration
config_q3 = model_bert_q3.config
```

```
# Total number of encoder layers
num_encoder_layers_q3 = config_q3.num_hidden_layers
```

```
# Number of attention heads per layer
num_attention_heads_q3 = config_q3.num_attention_heads
```

```
print(f"\n BERT Model ('{model_name_q3}') Details :- ")
print(f"Total number of encoder layers: {num_encoder_layers_q3}")
print(f"Number of attention heads per layer: {num_attention_heads_q3}")
```

```
 BERT Model ('bert-base-uncased') Details :-
Total number of encoder layers: 12
Number of attention heads per layer: 12
```

Q4. Tokenize and Decode a Sentence using BERT Tokenizer

```
model_name_q4 = "bert-base-uncased"
sentence_q4 = "Transformers make NLP tasks easier."
```

```
# Loading the bert-base-uncased tokenizer
print(f"Loading tokenizer: '{model_name_q4}' ...")
tokenizer_bert_q4 = AutoTokenizer.from_pretrained(model_name_q4)
print(f"Tokenizer '{model_name_q4}' loaded successfully.")
```

```
Loading tokenizer: 'bert-base-uncased' ...
Tokenizer 'bert-base-uncased' loaded successfully.
```

```
print(f"\nOriginal sentence: \"{sentence_q4}\"")
```

```
Original sentence: "Transformers make NLP tasks easier."
```

```
# Tokenizing the sentence
encoded_input_q4 = tokenizer_bert_q4(sentence_q4, return_tensors=None, add_special_tokens=True)
```

```
# Printing the token ids
token_ids_q4 = encoded_input_q4['input_ids']
print(f"\n1) Token IDs: {token_ids_q4}")
```

```
1) Token IDs: [101, 19081, 2191, 17953, 2361, 8518, 6082, 1012, 102]
```

```
# 2) Printing the decoded tokens
decoded_tokens_q4 = tokenizer_bert_q4.convert_ids_to_tokens(token_ids_q4)
print(f"\n2) Decoded Tokens: {decoded_tokens_q4}")
```

```
2) Decoded Tokens: ['[CLS]', 'transformers', 'make', 'nl', '##p', 'tasks', 'easier', '.', '[SEP]']
```